Decision trees and forests

- A simple, powerful, explainable classification model.
- Works for binary and multi-label classification.
- Also works for regression.
- Can be used in progressive ensemble mode through decision forests.
- Performs very well in a wide range of applications. Decision forests do quite well even on high dimensional data .

Decsion tree

- A DT is a multi-stage decision process that looks at a subset of attributes of the data vector at each stage.
- It starts with $\mathcal{L}$ at the root and at each stage the learning set is partitioned into two (typical) or more mutually exclusive sets at the child nodes based on conditional expressions.
- This process of splitting continues until some stopping criterion is met.
- The leaf nodes either have an associated label prediction function or a probability distribution over the labels which can be used to predict the label or the dependent variable value in case it is a regression.
- A query vector traverses the tree using the conditional expressions and ends up at a leaf. It is either given a label using the label prediction function at that leaf or the probability distribution at the leaf is output.

Example

A hypothetical school uses the following attributes to decide whether or not to give admission to an applicant.

1. Parental income in lakhs/month (I):
   a) $I > 2$ b) $1 < I \leq 2$ c) $0.5 < I \leq 1$ d) $I \leq 0.5$
2. Parents' education: (E)
   a) Both are graduates. b) Only one is a graduate. c) Neither is a graduate.
3. Score on test (S):
   a) ≥ 80 b) $60 \leq S < 80$ c) $50 \leq S < 60$ d) < 50

Example data set

i- income, e- Parents' education, s- test score.
a), b), c), d) refer to the attributes as defined above.

Student ID	**Income**	**Parents' ed.**	**Test score**	**Admitted?**
s1	ia)	ec)	sd)	Y
s2	id)	ec)	sa)	Y
s3	ic)	eb)	sb)	Y
s4	ib)	eb)	sb)	Y
s5	ic)	ea)	sc)	Y
s6	id)	ec)	sb)	N
s7	ib)	eb)	sd)	N
s8	ic)	ec)	sc)	N
s9	id)	ea)	sc)	N

First decision tree

FIRST TREE (Fully grown. Only nodes with non-null set of vectors shown. Vertical line indicates next level.)

```
root : {s1 to s9}
     |test = sa): {s2} Y
     |test = sb): {s3,s4,s6}
                |test education = eb): {s3,s4} Y
                |test education = ec): {s6} Y
     |test = sc): {s5,s8,s9}
                |test education=ea): {s5,s9}
                                    |test income=ic): {s5} Y
                                    |test income=id): {s9} N
                |test education=ec): {s8} N
     |test = sd): {s1,s7}
                |test income = ia): {s1} Y
                |test income = ib): {s7} N
```

Figure: One possible decision tree from the data above

Second decision tree

SECOND TREE (Not fully grown. Leaf nodes have probability distribution and a label given by highest probability label.)

```
root :
     |test=(sc): {s5,s8,s9} (Y=1/3, N=2/3) N
     |test!=(sc): {s1-s4,s6,s7} (Y=2/3, N=1/3) Y
```

Figure: Another possible decision tree from the given data. Often called weak learner, stump.

Abstract algorithm for decision tree learning

Algorithm 0.5: DTLEARN($\mathcal{L}$)

```
tree ← (root, L); //Each node is associated with a subset of L. The root is associated with L.
queue ← queue.add(root);
while (queue ≠ empty)
  do
  { node ← queue.head(); // head removes and returns first element of queue
    if (splitNode?(node))//splitNode? is defined later
    { childNodes ← createChildNodes(node);
      /* createChildNodes is a complex function which finds the 'best' split and the test condition for each child.
      Each condition is non-overlapping.*/
      tree ← growTree(tree, childNodes);
      /* Adds each child to the tree and associates the relevant subset of L with the child node.*/
      queue ← queue.add(childNodes); //queue up children for expansion
    }
    else giveLabel(node);
    /*Node cannot be split so a leaf. Each leaf has a label prediction function or an associated probability
    distribution.*/
  }
return (tree);
```

Algorithm for testing

Algorithm 0.6: DTTEST(*tree*, **x**)

```
node ← tree.root();
while (not leaf?(node))
  do
  { node ← descent(node, x);
    /* descend returns the child that satisfies the test condition w.r.t. to x. */
return (node.label())// return the label or the distribution.
```

Important variables in constructing a decision tree

1. Which attribute(s) to use for splitting?
2. What is the nature of the splitting criterion?
3. How many children should a node have? Binary versus n-ary.
4. When should a node not be split or when to stop splitting?
5. If a leaf node contains labels from different categories then what do we do?
6. How does one handle missing data while construting a tree and while testing?

How many children should a node have?

- A multiway tree can be converted to a binary tree.
- If $\mathbf{x}_{(i)}$ the ith feature has say k values $v_1, \ldots, v_k$, then we can get an equivalent binary tree of depth k with the tests $\mathbf{x}_{(i)} = v_j$ and $\mathbf{x}_{(i)} \neq v_j$. So, if k is not very large we can use binary splits.
- Mostly applications only use binary splits. Experiments have shown that n-ary splits do not give significant extra accuracy.
- For nominalized features or those with a few discrete values multi-way splits with children equal to the number of values can be used.

How to give leaf labels?

- One simple approach is to use Bayes rule: pick the label with the highest probability.
- More generally: Let $\lambda(\omega_j, \omega_i)$ be the risk/loss of assigning ω_i when the actual label is ω_j. Let t be a leaf node and $L_t \subseteq \mathcal{L}$ be the subset of vectors at t then the label function is:

$$\underset{\omega_k}{\operatorname{argmin}} \sum_{\mathbf{x}_j \in L_t} \lambda(y_j, \omega_k), \quad y_j \text{ is the actual label of } \mathbf{x}_j$$

  We choose a label that minimizes the total loss or risk.
- For $0-1$ loss:

$$\lambda(\omega_i, \omega_j) = \begin{cases} 1 & i \neq j \\ 0 & i = j \end{cases}$$

  This means choose the most popular label at t.

Choosing the attribute for splitting

- A node where all the associated vectors have the same label is called a pure node. Such a node is never split and is a leaf. A node that is not pure is impure.
- The goal of decision tree creation for classification is to reach pure nodes as quickly as possible. So, one strategy (greedy heurisitc) is to choose a split that leads to <u>maximum impurity reduction</u> between parent and children
- To quantify purity let $P(\omega_j|t)$ be the probability that a query vector at node t will get label ω_j. Assuming C classes let $\mathbf{z} = (P(\omega_1|t), \ldots, P(\omega_C|t))$. Clearly $z_i \geq 0$ and $\sum_{i=1}^{C} z_i = 1$. $P(\omega_i|t) = \frac{N_i(t)}{N(t)}$, where $N_i(t)$ is the number of vectors with label ω_i at node t and $N(t)$ is the total number of vectors at node t.
- Define the *impurity function* $\phi : \mathcal{Z} \to [0, 1]$, where $\mathcal{Z}$ is the set of all possible $\mathbf{z}$ vectors.

Properties of the impurity function ϕ

- $\phi(\mathbf{z})$ is 0 only when $z_i = 1$ for some i and $z_j = 1$ for $i \neq j$. That is all vectors at the node have the same label. It is a pure node.
- $\phi(\mathbf{z})$ is a maximum when $z_i = \frac{1}{C}$, $\forall i$. There is maximum uncertainty about the label at the node. The impurity is a maximum. All labels are equally represented.
- ϕ is a symmetric function of $z_1, \ldots, z_C$. Using the previous property this means it will attain a maximum at $z_i = \frac{1}{2}$ for all i.
- We often write $\phi(t)$ (instead of $\phi(\mathbf{z})$) for impurity at node t. Note that each node t has a corresponding $\mathbf{z}$.

Possible impurity functions

Some functions that satisfy the properties for impurity functions are:

- **Gini impurity/index**: $\phi(t) = \sum_{i \neq j} P(\omega_i|t)P(\omega_j|t) = 1 - \sum_{i=1}^{C} P(\omega_i|t)^2$.
- **Entropy**: $\phi(t) = -\sum_{i=1}^{C} P(\omega_i|t) log_2(P(\omega_i|t))$ - often called Shannon entropy.
- **Classification error**: $\phi(t) = 1 - \max_i[P(\omega_i|t)]$.

Plot of different impurity functions for $C = 2$

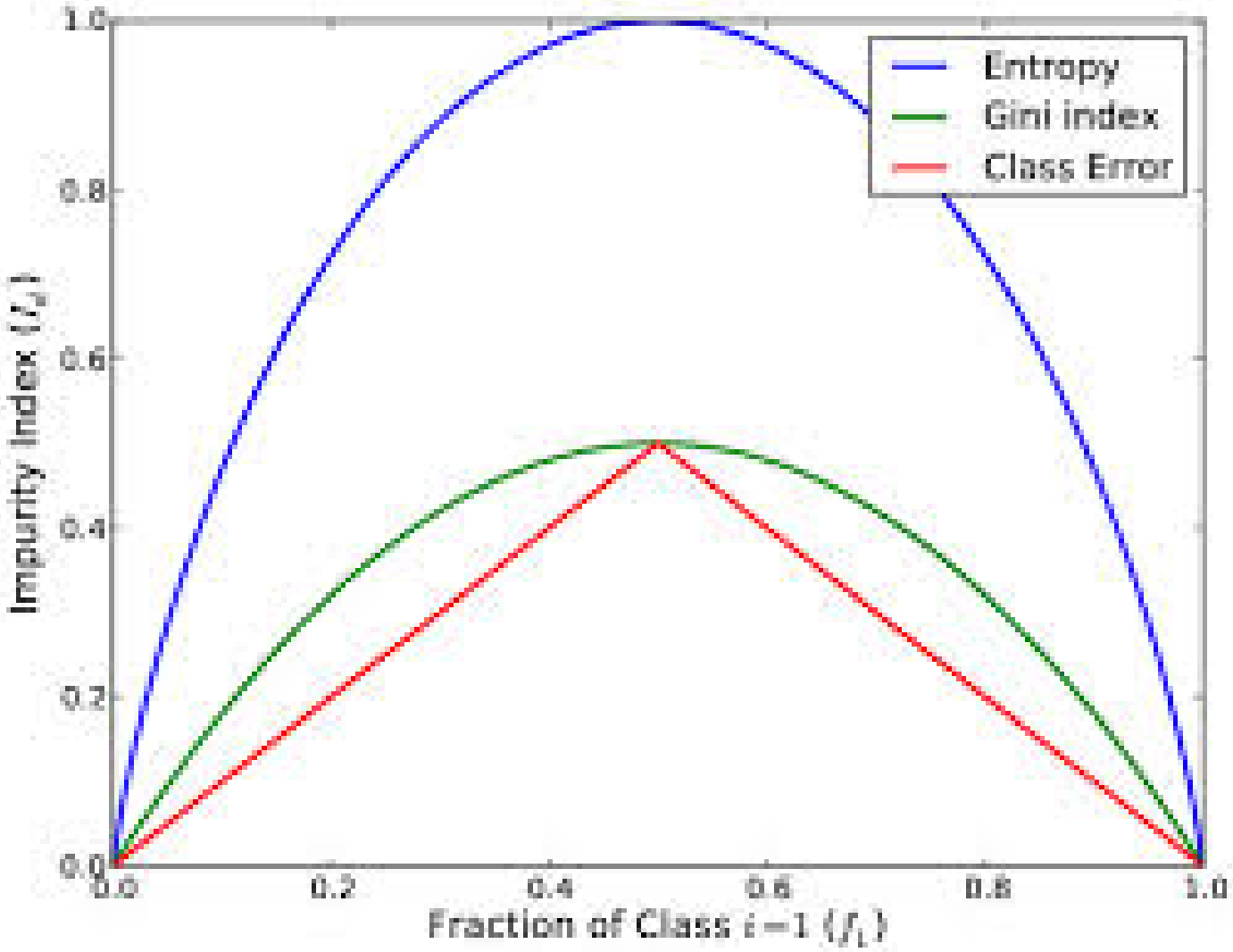

Figure: A plot of different impurity functions (source: inspirehep.net)

Choice of attribute to split

- Assuming we split only on a single attribute we split on attribute a_p that gives the maximum reduction in impurity. t is the parent node and t_i refer to the child nodes of t. $N(t)$ is number of vectors at node t and $N(t_i)$ are the number of vectors at node t_i. The attribute a_p is chosen by:

$$\max_{a_p} \Delta(a_p, t) = \phi(t) - \sum_{i \in \text{children}} \phi(t_i) \frac{N(t_i)}{N(t)}$$

Complexity of finding the best split

The number of splits to be investigated at a node to find the best attribute for a split depends on the type of values for an attribute.

- Nominal: Number of possible splits to investigate are all possible subsets of values so can be $2^{(\text{no. of values})}$.
- Ordinal: Values are ordered (e.g. grades A, B, C, D, E, F). Number of splits to to consider can again be large but may have to maintain ordering constraints e.g. $\{A, E\}$, $\{B, C, D, F\}$ may be illegal.
- Discrete: values are finite and discrete. Similar to nominals so possibilities are large.
- Continuous: condition can be on a single feature or a combination of features. Multi-way splits are possible with different conditions labelling each edge. Most often binary splits are used where a threshold is used to decide the two-way split.

  Another way is to learn a linear separator on the vectors at the node. This separator need not be parallel to any of the axes.

Handling inherent bias of multi-way splits

- Higher branching nodes are favoured for impurity reductions. For example, if an attribute value is unique for an instance (e.g. roll no.) then we can get children with just one vector and each child will have 0 impurity. But this is not the best choice for classification.
- Bias is addressed by using a scale factor calculated by the number of vectors at a child without considering labels. For e.g. above if parent has B nodes and degree of branching is B then the entropy due to branching is $-\sum_{i=1}^{B} P(t_i) log_2(P(t_i))$. Since every child has just 1 vector $P(t_i) = \frac{1}{B}$ giving $-\frac{1}{B} \times log_2(\frac{1}{B}) \times B = log_2(B)$. So, $\Delta\phi(t)$ is divided by $log_2(B)$.

When to stop splitting

- Apply a stopping criterion: i) Use a threshold. If $\Delta\phi(t)$ is below a pre-defined threshold ii) impose a limit on the number of vectors at a node. Method i) can lead to horizon effects.
- Grow the tree fully and then prune.

Pruning

Let T_t be the tree hanging from node t and $\epsilon(t)$, $\epsilon(T_t)$ be the estimated error rates at t and T_t respectively.

Prune T_t if $\epsilon(t) \leq \epsilon(T_t)$

. $\epsilon(t) = \frac{M_t}{N_t}$ where M_t is number of misclassified vectors at node t. Similarly, $\epsilon(T) = \sum_{t \in T_\ell} \epsilon(t)$ where T_ℓ is the set of leaves of T_t

To incorporate complexity of the model let $\alpha \in \mathbb{R}$, $\alpha > 0$. Define $\epsilon_\alpha(t) = \epsilon(t) + \alpha$ and $\epsilon_\alpha(T) = \epsilon(T) + |T_\ell|\alpha$.

Prune if $\epsilon_\alpha(t) \leq \epsilon_\alpha(T_t)$

.

Pruning can also be done by testing the pruned and un-pruned trees on a validation set.

Handling missing data

- Missing data in the form of missing attribute values must be addressed both during tree construction and when testing a query vector.
- During tree construction:
  - Remove vectors with missing values from the learning set - this is wasteful so not a desirable method.
  - Exclude the vector from the impurity calculation if the value for the splitting attribute is missing.
  - Provide a value for the missing attribute value.
    - Most common value.
    - The median value.
    - The average value.
- For testing: provide a value for the missing attribute.

Handling missing values: surrogate splits

- During tree construction at each node we create surrogate or back-up splits (one or more). The surrogate splits can be chosen in terms of a) descending $\Delta(t, a_t)$ or b) closeness of distribution of labels at child nodes to the distribution of labels of children of the primary split.
- During test if an attrbute value is not present we use a surrogate split.

Decision (or Random) forests

- A random forest is a set of decision trees constructed using a source of randomness which is used to inject randomness when needed during tree construction.
- The same randomness source is used to construct all the trees and whenever a random value is needed it is drawn i.i.d from the random source.
- The label of any query vector $\mathbf{x}$ is calculated by combining the output of each tree in the forest. In the simplest case assuming each leaf in each tree has a label generating function the forest assigns the most frequent label.
- Random forest construction and testing can be completely parallelized.

Factors to consider in constructing random forest

- How many trees should be in the forest? We represent this by K.
- At what points do we inject randomness in the construction?
- Which algorithm to use to construct the individual trees in the forest?

How many trees in forest?

Computation effort is directly proportional to K so we want K to be as small as possible. The value of K does depend on the nature of trees in the forest. But in most cases we see the behaviour below:

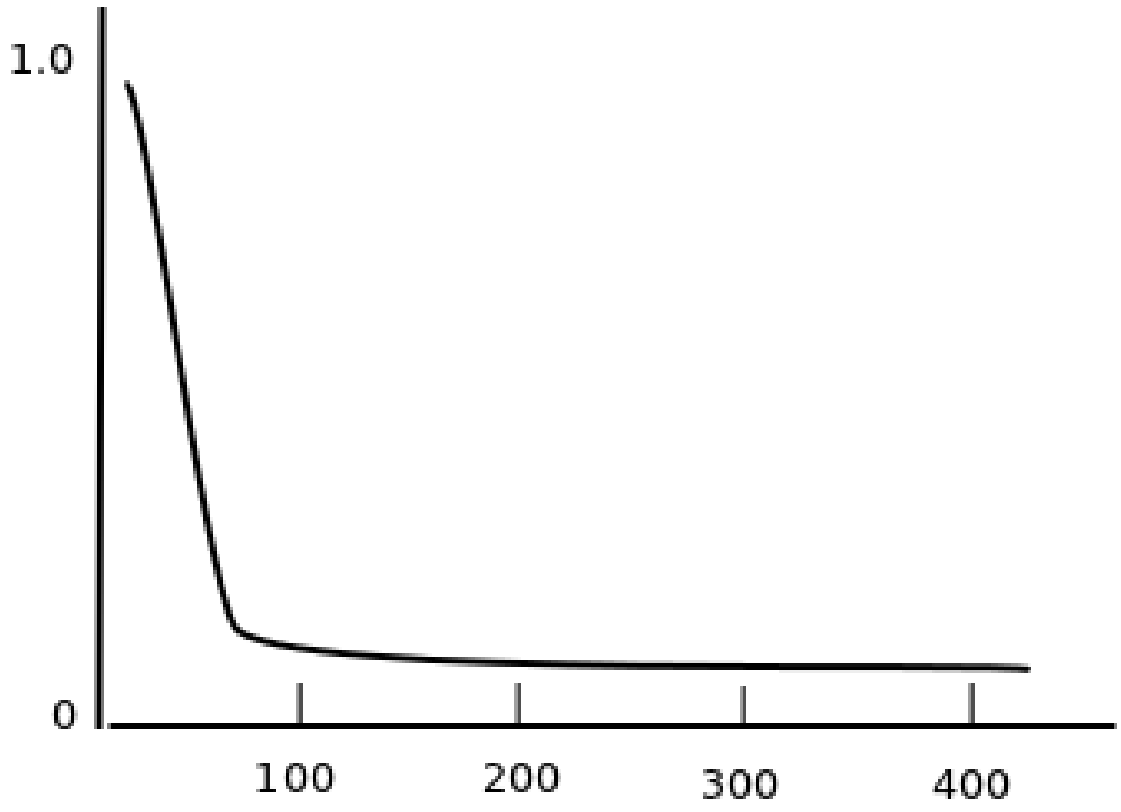

Figure: Variation of error on a test set with K (size of forest).

Observations on variations with K

- K larger than the threshold at which the curve flattens is not useful.
- Note that large K does not lead to overfitting.
- Choose K equal to the threshold at which error has reached an approximately stable value.
- Due to randomness different forests of size K on the same data set will show slightly different error rates but the differences usually are not statistically significant other factors remaining the same.

Injection of randomness

Randomness enters tree construction at one, two or sometimes three points.

- Choice of random subset of $\mathcal{L}$ used for training. In some approaches all trees are constructed using using $\mathcal{L}$.
- Choice of splitting features at a node.
- Choice of subset or threshold - that is splitting criterion. This choice may not exist if tree is constructed using the greedy approach of maximum reduction in impurity.

Random subset of $\mathcal{L}$ for training

- The random training sets $\mathcal{L}_i$ for training a tree in a forest can be created by *bootstrap aggregation* or *bagging*.
- $|\mathcal{L}|$ vectors are chosen uniformly randomly from $\mathcal{L}$ <u>with replacement</u>.
- This means the average size of the bagged learning set $\mathcal{L}_i$ is approx. $0.632 \cdot |\mathcal{L}|$.
- The set of vectors $\mathcal{L}_{OOB} = \mathcal{L} - \mathcal{L}_i$ called the out of bag or OOB vectors can be used for validation, testing during tree construction or parameter tuning.

Choice of features for node splits

- The number of attributes considered for splitting at each node (represented by m) is usually fixed to a constant value between 1 and $<$ *small integer* $>$. The value of m can be tuned using the OOB vectors.
- At each node m features are chosen randomly from the set of features (that is d sized set). In principle the value of m can also be randomly chosen either at each node in a tree or for each tree. However, in practice a fixed value is chosen for all trees in the forest.
- The choice of splitting criterion like threshold/subsets can also be randomized (most easily for binary decision trees). In practice this is often not done since it adds to the complexity of tree construction and increases K.

Choice of tree construction

- One common choice is to grow the tree to the maximum extent.
- Another extreme choice is to use a depth one tree (called a stump) - this usually means K is larger. Often instead of a stump the maximum depth is fixed. Used in boosting ensemble methods.
- Put a limit on the number of vectors at terminal nodes - typical numbers range from 2 to 5.
- A fourth possibility is to grow and prune - as in the case of standard single decision tree where there is no randomness.

Pseudocode for random forest construction

Algorithm 0.7: $\textsc{RandomForest}(\mathcal{L}, K, m)$

comment: K - no. of trees; m - no. of attributes for split at a node

forest $\leftarrow$ *null*; **comment:** *forest* is a set of trees.

for $i \leftarrow 1$ **to** K
  do
  $\mathcal{L}_i \leftarrow$ Create training set by bagging;
  tree $\leftarrow$ *createInitialTree*$(\mathcal{L}_i)$;
  comment: Initial tree with all vectors in $\mathcal{L}_i$ at the root.
  Grow the *tree* using choice of tree construction alg. For splitting at each node choose m features uniformly randomly at; each node and find the best split over these m features.
  forest $\leftarrow$ *forest* $\cup$ *tree*;
return (*forest*);

Algorithm RandomForest to construct a random forest

Convergence

- Let the forest $\mathbb{F} = \{T_i | i = 1..K\}$ of K decision trees. The learning set $\mathcal{L}$ is drawn iid from the joint distribution $(\mathbf{X}, Y)$ of feature space $\mathbf{X}$ and label space Y.
- Define the margin function with respect to $\mathbb{F}$ as:
  $mg(\mathbf{X}, Y) =$ (Average no. of votes for the correct label) $-$ (Average no. of votes for the most frequent label that is not correct)
  Formally:

Definition 3

$mg(\mathbf{X}, Y) = \underset{\mathbb{F}}{avg}\ I(T_i(\mathbf{X}) = Y) - \underset{j \neq Y}{max}\ \underset{\mathbb{F}}{avg}\ I(T_i(\mathbf{X} = Y) = j)$ *where I is the indicator function*

$$I(T_i(\mathbf{X} = Y) = \begin{cases} 1 & T_i(\mathbf{x}) = y,\ (\mathbf{x}, y) \in (\mathbf{X}, Y) \\ 0 & otherwise \end{cases}$$

- The larger the margin the more we are sure the predicted label is correct. We get an error when $mg(\mathbf{X}, Y) < 0$. So, generalization error ϵ is given by: $\epsilon = P_{(\mathbf{X}, Y)}(mg(\mathbf{X}, Y) < 0)$

- For large K one can prove:[7]

Theorem 10

In $\mathbb{F}$ as $K \to \infty$ almost surely for all sequences $t_1, \ldots, t_K$, ϵ converges to

$$P_{\mathbf{X},Y}[P_t(T(\mathbf{X}, t) = Y) - \max_{j \neq Y} P_t(T(\mathbf{X}, t) = j) < 0]$$

This theorem tells us that random forests will not overfit as K increases and why it reaches a stable value of generalization error.

[7] Leo Breiman, Random Forests, Machine Learning, 45, 5-32, 2001.

Factors affecting generalization error

- The forest error rate depends on two parameters:
  - First, the strength of the trees represented by s. This measures how 'good' on average individual trees are.
  - Second, the correlation between trees represented by ρ. This measures how similar two trees are in $\mathbb{F}$.
- Increasing m (no. of attributes considered for splitting) increases both s and ρ and decreasing it decreases both. So, the right choice of m is important. In practice small values of m (1 to 5) work well though Breiman recommends upto $\sqrt{d}$.
- Breiman derives the following bound for the generalization error

$$\epsilon \leq \frac{\bar{\rho}(1 - s^2)}{s^2}$$

While this bound is probably loose in most cases the bound is expressed in terms of $\bar{\rho}$ (average correlation) and s (strength).

- Formally, a margin function is defined as:
  $mr(\mathbf{X}, Y) = P_\theta(T(\mathbf{X}, \theta) = Y) - \max_{j \neq Y} P_\theta(T(\mathbf{X}, \theta) = j)$.

  Note that $mg(\mathbf{X}, Y)$ defined earlier is for a given forest $\mathbb{F}$.

  The strength s is defined as the expectation
  $s = E_{\mathbf{X},Y}[mr(\mathbf{X}, Y)]$.

  If $s > 0$ then using Chebyshev's inequality we get $\epsilon \leq \frac{Var(mr)}{s^2}$.

  We can show that $Var(mr) \leq \bar{\rho} E_\theta[Var(\theta)]$ and that
  $Var(\theta) \leq 1 - s^2$, $\bar{\rho}$ is the average correlation.

  Combining everything gives the generalization bound [8].

[8] For details see Leo Breiman, Random Forests, Machine Learning, 45, 5-32, 2001.